

Peter Petito

Z23712200

Homework 2 - COP4045

Github username: ppetito, all files inside homework 2 directory

Problem 1:

Code:

```
# Peter Petito - Homework 2
# Problem 1
import ast
import io
import tokenize
from typing import Tuple

def line_number(source_filename: str, target_filename: str) -> None:
    """Write each line of source_filename to target_filename, prefixed with its line
    number"""
    try:
        with open(source_filename, 'r') as infile:
            lines = infile.readlines()
        with open(target_filename, 'w') as outfile:
            for i, line in enumerate(lines, start=1):
                outfile.write(f"{i}. {line}")
                if not line.endswith('\n'):
                    outfile.write('\n')
    except Exception as error:
        print(f"Error numbering lines from '{source_filename}' to '{target_filename}':
{error}")
        raise

def _strip_comments_and_blanks(code: str) -> str:
    lines = code.split('\n')
    try:
        for tok in tokenize.generate_tokens(io.StringIO(code).readline):
            if tok.type == tokenize.COMMENT:
                row, col = tok.start
                lines[row - 1] = lines[row - 1][:col].rstrip()
    except tokenize.TokenizeError:
        pass
    lines = [ln for ln in lines if ln.strip()]
```

```

    return '\n'.join(lines) + '\n' if lines else ''

def _args_string(node: ast.FunctionDef) -> str:
    a = node.args
    parts = [x.arg for x in getattr(a, 'posonlyargs', [])] + [x.arg for x in a.args]
    if a.vararg:
        parts.append('*' + a.vararg.arg)
    parts += [x.arg for x in a.kwonlyargs]
    if a.kwarg:
        parts.append('*' + a.kwarg.arg)
    return ', '.join(parts)

def parse_functions(filename: str) -> Tuple[Tuple[int, str, str, str], ...]:
    """Return (line_no, name, args, code) for each top level function in filename,
    sorted by name."""
    try:
        with open(filename, 'r') as f:
            source = f.read()
        tree = ast.parse(source, filename=filename)
        src_lines = source.split('\n')
        functions = []
        for node in tree.body:
            if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                end = getattr(node, 'end_lineno', node.lineno)
                raw = '\n'.join(src_lines[node.lineno - 1:end]) + '\n'
                functions.append((node.lineno, node.name, _args_string(node),
                _strip_comments_and_blanks(raw)))
        functions.sort(key=lambda t: t[1])
        return tuple(functions)
    except Exception as error:
        print(f"Error parsing '{filename}': {error}")
        raise

def main() -> None:
    this_file = __file__
    out_file = "p1_line_numbers_output.txt"
    line_number(this_file, out_file)
    print(f"Part a): numbered lines written to '{out_file}'.\n")

```

```

print("Part b): functions (sorted alphabetically by name):")
for func in parse_functions(this_file):
    print(func)

if __name__ == "__main__":
    main()

```

Output:

```

# Peter Petito - Homework 2
# Problem 1
import ast
import io
import tokenize

(35, '_args_string', 'node', "def _args_string(node: ast.FunctionDef) -> str:\n    a = node.args\n    parts = [x.arg for x in\n    getattr(a, 'posonlyargs', [])] + [x.arg for x in a.args]\n    if a.vararg:\n        parts.append('*' + a.vararg.arg)\n    parts += [x.arg for x in a.kwonlyargs]\n    if a.kwarg:\n        parts.append('*' + a.kwarg.arg)\n    return ', '.join(parts)\n")
(22, '_strip_comments_and_blanks', 'code', "def _strip_comments_and_blanks(code: str) -> str:\n    lines = code.split('\n')\n    try:\n        for tok in tokenize.generate_tokens(io.StringIO(code).readline):\n            if tok.type == tokenize.COMMENT:\n                row, col = tok.start\n                lines[row - 1] = lines[row - 1][:col].rstrip()\n            except tokenize.TokenizeError:\n                pass\n    lines = [ln for ln in lines if ln.strip()]\n    return '\n'.join(lines) + '\n' if lines else ''")
(7, 'line_number', 'source_filename, target_filename', "def line_number(source_filename: str, target_filename: str) -> None:\n    \"\"\"Write each line of source_filename to target_filename, prefixed with its line number\"\"\"\n    try:\n        with open(source_filename, 'r') as infile:\n            lines = infile.readlines()\n            with open(target_filename, 'w') as outfile:\n                for i, line in enumerate(lines, start=1):\n                    outfile.write(f'{i}. {line}')\n    except Exception as error:\n        print(f"Error numbering lines from '{source_filename}' to '{target_filename}': {error}")\n        raise\n")
(66, 'main', '', "def main() -> None:\n    this_file = __file__\n    out_file = 'p1_line_numbers_output.txt'\n    line_number(this_file, out_file)\n    print(f"Part a): numbered lines written to '{out_file}'.\\n")\n    print("Part b): functions\n    (sorted alphabetically by name):")\n    for func in parse_functions(this_file):\n        print(func)\n")
(46, 'parse_functions', 'filename', "def parse_functions(filename: str) -> Tuple[Tuple[int, str, str, str], ...]:\n    \"\"\"Return (line_no, name, args, code) for each top level function in filename, sorted by name.\"\"\"\n    try:\n        with open(filename, 'r') as f:\n            source = f.read()\n            tree = ast.parse(source, filename=filename)\n            src_lines = source.split('\n')\n            functions = []\n            for node in tree.body:\n                if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):\n                    end = getattr(node, 'end_lineno', node.lineno)\n                    raw = '\n'.join(src_lines[node.lineno - 1:end]) + '\n'\n                    functions.append((node.lineno, node.name, _args_string(node), _strip_comments_and_blanks(raw)))\n            functions.sort(key=lambda t: t[1])\n            return tuple(functions)\n    except Exception as error:\n        print(f"Error parsing '{filename}': {error}")\n        raise\n")

[Done] exited with code=0 in 0.039 seconds

```

Problem 2: Comprehensions

Code:

```

# Peter Petito - Homework 2
# Problem 2: Comprehensions

def main():
    # a) a,b,c,d distinct in [1,10], a^2+b^2 = c^2+d^2

    result_a = [(a, b, c, d)
                 for a in range(1, 11) for b in range(1, 11)
                 for c in range(1, 11) for d in range(1, 11)

```

```

        if len({a, b, c, d}) == 4 and a**2 + b**2 == c**2 + d**2]

print("a)", result_a)

# b) (lowercase, length) for strings shorter than 5 chars
words_b = ['One', 'SEVEN', 'three', 'two', 'Ten']
result_b = [(w.lower(), len(w)) for w in words_b if len(w) < 5]
print("b)", result_b)

# c) "Firstname Middlename Lastname" -> "Firstname M. Lastname"
names = ['Christopher Ashton Kutcher', 'Elizabeth Stamatina Fey']
result_c = [f"{n.split()[0]} {n.split()[1][0]}. {n.split()[2]}" for n in names]
print("c)", result_c)

# d) anagram pairs (case insensitive) between lst1 and lst2
lst1 = ["Spam", "Trams", "Elbows", "Tops", "Astral"]
lst2 = ["Bowels", "Sample", "Altars", "Stop", "Course", "Smart"]
result_d = [(w1, w2) for w1 in lst1 for w2 in lst2 if sorted(w1.lower()) ==
sorted(w2.lower())]
print("d)", result_d)

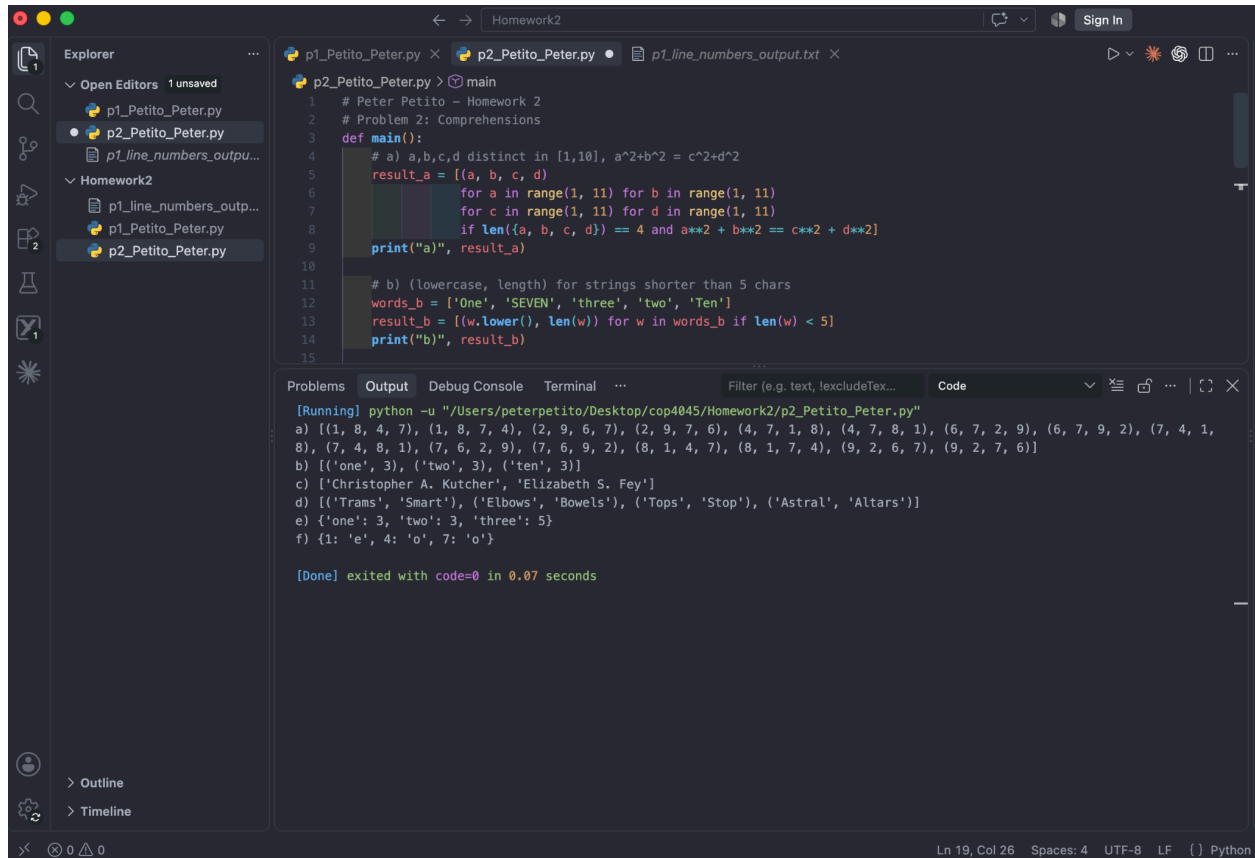
# e) string -> length
s = ['one', 'two', 'three']
result_e = {word: len(word) for word in s}
print("e)", result_e)

# f) index -> vowel character (case insensitive)
text = "Hello world"
result_f = {i: c for i, c in enumerate(text) if c.lower() in 'aeiou'}
print("f)", result_f)

if __name__ == "__main__":
    main()

```

Output:



Problem 3: Social Network

Code:

```

# Peter Petito - Homework 2
# Problem 3: Social Network

import csv
from typing import Dict, List, Tuple

SocialNetwork = Dict[str, Tuple[str, List[str]]]

def add_user(sn: SocialNetwork, username: str, fullname: str):
    """Add a new user with no friends to the sn. Return True if added, false if the
    username
    exists."""
    try:
        if username in sn:
            return False
        sn[username] = (fullname, [])
        return True

```

```

except Exception as error:
    print(f"Error adding user '{username}': {error}")
    raise

def add_friend(sn: SocialNetwork, user1: str, user2: str):
    """Add a mutual friend link between user1 and user2 in sn. Return True if success,
False
otherwise."""
    try:
        if user1 not in sn or user2 not in sn:
            return False
        if user2 not in sn[user1][1]:
            sn[user1][1].append(user2)
        if user1 not in sn[user2][1]:
            sn[user2][1].append(user1)
        return True
    except Exception as error:
        print(f"Error adding friend link between '{user1}' and '{user2}': {error}")
        raise

def get_friends(sn: SocialNetwork, user1: str, distance: int):
    """Return all friends of user1 within the given link distance (BFS)"""
    try:
        if user1 not in sn:
            return []
        visited = {user1}
        current_level = [user1]
        result = []
        for _ in range(distance):
            next_level = []
            for u in current_level:
                for f in sn[u][1]:
                    if f not in visited:
                        visited.add(f)
                        next_level.append(f)
                        result.append(f)
            current_level = next_level
            if not current_level:
                break
        return result
    except Exception as error:
        print(f"Error getting friends of '{user1}': {error}")

```

```

        raise

def save_network(filename: str, sn: SocialNetwork):
    """Save a social network dictionary to a CSV file: username, fullname,
    semicolon-separated friends."""
    try:
        with open(filename, 'w', newline='') as f:
            writer = csv.writer(f)
            for username, (fullname, friends) in sn.items():
                writer.writerow([username, fullname, ';' + join(friends)])
    except Exception as error:
        print(f"Error saving network to '{filename}': {error}")
        raise

def load_network(filename: str):
    """Load and return a social network dictionary from a CSV file saved with
    save_network()."""
    try:
        sn: SocialNetwork = {}
        with open(filename, 'r', newline='') as f:
            reader = csv.reader(f)
            for row in reader:
                username, fullname, friends_str = row
                friends = friends_str.split(';') if friends_str else []
                sn[username] = (fullname, friends)
        return sn
    except Exception as error:
        print(f"Error loading network from '{filename}': {error}")
        raise

def main():
    sn: SocialNetwork = {
        'alice': ('Alice Smith', ['maria']),
        'maria': ('Maria Cortez', ['alice', 'joe', 'david']),
        'joe': ('Joseph Adams', ['maria', 'eve']),
        'eve': ('Evelyn Cooper', ['joe']),
        'david': ('David Benson', ['maria']),
    }

    print("a) add_user:")
    print("  add new user 'frank':", add_user(sn, 'frank', 'Frank Ortiz'))

```

```

print("  add existing user 'alice':", add_user(sn, 'alice', 'Alice Smith'))

print("b) add_friend:")
print("  link frank-eve:", add_friend(sn, 'frank', 'eve'))
print("  link frank-unknown:", add_friend(sn, 'frank', 'unknown'))
print("  sn:", sn)

print("c) get_friends:")
print("  alice, distance 1:", get_friends(sn, 'alice', 1))
print("  alice, distance 2:", get_friends(sn, 'alice', 2))
print("  unknown user:", get_friends(sn, 'nobody', 2))

print("d-e) save_network / load_network:")
save_network('social_network.csv', sn)
loaded = load_network('social_network.csv')
print("  loaded == original:", loaded == sn)
print("  loaded:", loaded)

if __name__ == "__main__":
    main()

```

Output:

```

p3_Petito_Peter.py > add_friend
1 # Peter Petito - Homework 2
2 # Problem 3: Social Network
3
4 import csv
5 from typing import Dict, List, Tuple
6
7 SocialNetwork = Dict[str, Tuple[str, List[str]]]
8
9 def add_user(sn: SocialNetwork, username: str, fullname: str):
10     """Add a new user with no friends to the sn. Return True if added, false if the username
11     exists."""
12     try:
13         if username in sn:

```

Problems Output Debug Console Terminal Ports Serial Monitor ... Filter (e.g. text, lexcludeTex... Code

```

[Running] python -u "/Users/peterpetito/Desktop/cop4045/Homework2/p3_Petito_Peter.py"
a) add_user:
  add new user 'frank': True
  add existing user 'alice': False
b) add_friend:
  link frank-eve: True
  link frank-unknown: False
  sn: {'alice': ('Alice Smith', ['maria']), 'maria': ('Maria Cortez', ['alice', 'joe', 'david']), 'joe': ('Joseph Adams', ['maria', 'eve']), 'eve': ('Evelyn Cooper', ['joe', 'frank']), 'david': ('David Benson', ['maria']), 'frank': ('Frank Ortiz', ['eve'])}
c) get_friends:
  alice, distance 1: ['maria']
  alice, distance 2: ['maria']
  unknown user: []
d-e) save_network / load_network:
  loaded == original: True
  loaded: {'alice': ('Alice Smith', ['maria']), 'maria': ('Maria Cortez', ['alice', 'joe', 'david']), 'joe': ('Joseph Adams', ['maria', 'eve']), 'eve': ('Evelyn Cooper', ['joe', 'frank']), 'david': ('David Benson', ['maria']), 'frank': ('Frank Ortiz', ['eve'])}

[Done] exited with code=0 in 0.092 seconds

```

Ln 27, Col 38 Spaces: 4 UTF-8 LF Python 3.11.4 Prettier

Problem 4:

Code:

```
# Peter Petito - Homework 2
# Problem 4

import csv
from typing import Dict, List, Tuple, Optional

CASTS_FIELDS = ['Title', 'Year', 'Director', 'Actor1', 'Actor2', 'Actor3', 'Actor4',
                'Actor5']

def load_csv_rows(filename: str, fieldnames: Optional[List[str]] = None) ->
List[dict]:
    """Read a CSV file and return its rows as a list of dicts (uses fieldnames if the
    file has no header)."""
    try:
        with open(filename, 'r', encoding='utf-8', newline='') as f:
            reader = csv.DictReader(f, fieldnames=fieldnames)
            return list(reader)
    except Exception as error:
        print(f"Error reading '{filename}': {error}")
        raise

def load_casts(filename: str) -> Dict[Tuple[str, str], Tuple[str, List[str]]]:
    """Return a dict mapping (title, year) -> (director, [actors]) from a top-casts CSV
    file (no header)."""
    rows = load_csv_rows(filename, fieldnames=CASTS_FIELDS)
    casts = {}
    for row in rows:
        key = (row['Title'], row['Year'])
        actors = [row[f'Actor{i}']] for i in range(1, 6) if row.get(f'Actor{i}')
        casts[key] = (row['Director'], actors)
    return casts

def display_ranking(ranking: List[tuple], limit: Optional[int] = None) -> None:
    """Print a ranking list (a list of tuples), optionally cut to the first 'limit'
    entries."""
    for entry in (ranking[:limit] if limit else ranking):
        print(entry)
```

```

def display_top_collaborations(topRated_file: str, casts_file: str, limit:
Optional[int] = None) -> List[Tuple[str, str, int]]:
    """
    Display the ranking of (director, actor, movie_count) for director and actor pairs
    that collaborated on movies in the top-rated list, ordered by descending count.
    """
    try:
        casts = load_casts(casts_file)
        topRated = load_csv_rows(topRated_file)

        counts: Dict[Tuple[str, str], int] = {}
        for movie in topRated:
            key = (movie['Title'], movie['Year'])
            if key in casts:
                director, actors = casts[key]
                for actor in actors:
                    pair = (director, actor)
                    counts[pair] = counts.get(pair, 0) + 1

        ranking = [(director, actor, count) for (director, actor), count in
counts.items()]
        ranking.sort(key=lambda entry: entry[2], reverse=True)
        display_ranking(ranking, limit)
        return ranking
    except Exception as error:
        print(f"Error computing top collaborations: {error}")
        raise

def display_top_actors(top_grossing_file: str, casts_file: str, limit: Optional[int] =
None) -> List[Tuple[str, int]]:
    """
    Display the ranking of actors from the top-grossing list ordered by the
    total USA box office (descending) of the movies they acted in.
    """
    try:
        casts = load_casts(casts_file)
        top_grossing = load_csv_rows(top_grossing_file)

        totals: Dict[str, int] = {}

```

```

        for movie in top_grossing:
            key = (movie['Title'], movie['Year'])
            if key in casts:
                box_office = int(movie['USA Box Office'])
                _, actors = casts[key]
                for actor in actors:
                    totals[actor] = totals.get(actor, 0) + box_office

    ranking = list(totals.items())
    ranking.sort(key=lambda entry: entry[1], reverse=True)
    display_ranking(ranking, limit)
    return ranking
except Exception as error:
    print(f"Error computing top actors: {error}")
    raise

def main() -> None:
    top Rated file = 'imdb-top-rated.csv'
    top_grossing_file = 'imdb-top-grossing.csv'
    casts_file = 'imdb-top-casts.csv'

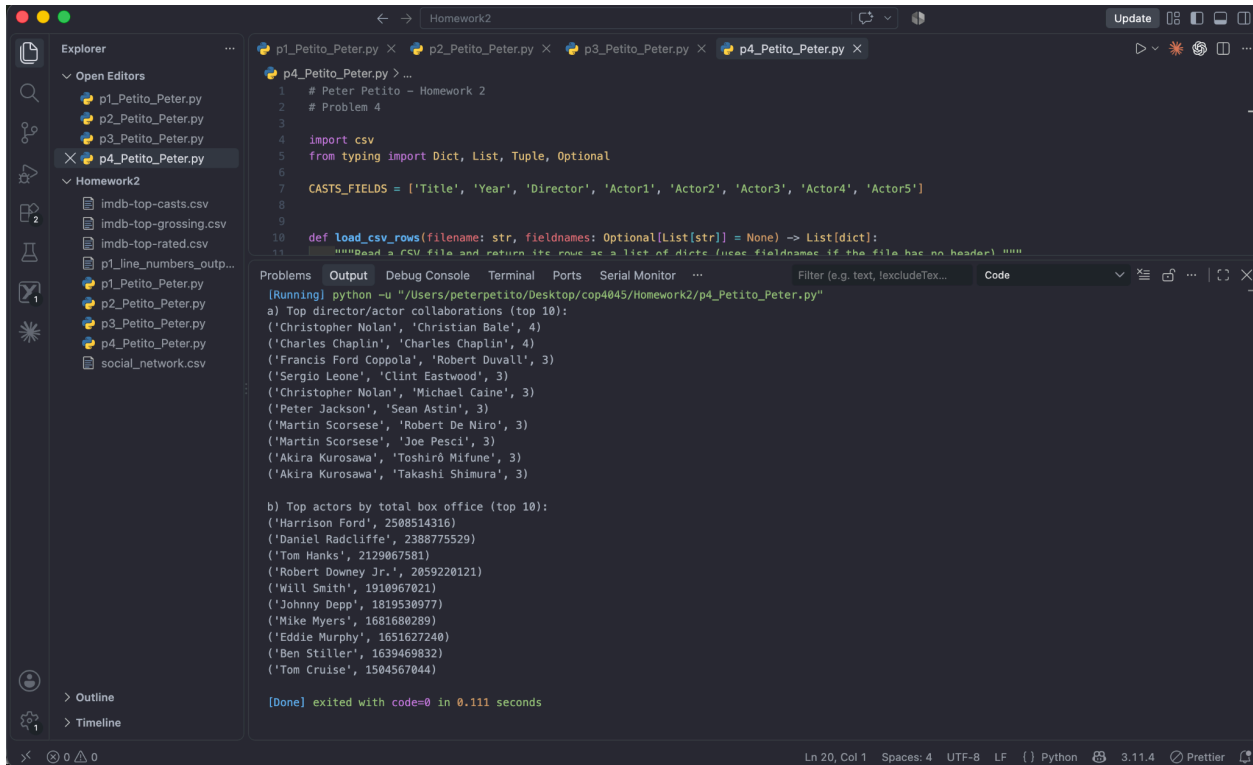
    print("a) Top director/actor collaborations (top 10):")
    display_top_collaborations(top Rated file, casts_file, limit=10)

    print("\nb) Top actors by total box office (top 10):")
    display_top_actors(top_grossing_file, casts_file, limit=10)

if __name__ == "__main__":
    main()

```

Output:



Problem 5 - Weather Station Analyzer with coding agent help:

Code:

```

# Peter Petito - Homework 2
# Problem 5 - Weather Station Analyzer

import datetime
import sys

def read_observations(filename):
    """Read, validate, and organize weather-station observations.

    Args:
        filename: Path to a text file containing one observation per line in
            ``station,date,temperature`` format.

    Returns:
        A tuple containing a dictionary that maps station names to sorted lists
        of ``(date, temperature)`` tuples and a list of ``(line_number,
        error_message)`` tuples for invalid input lines.

    """

```

```

observations = {}
errors = []
seen_observations = set()

with open(filename, "r", encoding="utf-8") as observation_file:
    for line_number, line in enumerate(observation_file, start=1):
        fields = line.rstrip("\n").split(",")
        if len(fields) != 3:
            errors.append((line_number, "expected station,date,temperature"))
            continue

        station, date_text, temperature_text = (field.strip() for field in fields)

        try:
            date = datetime.datetime.strptime(
                date_text, "%I:%M:%S %p %m/%d/%Y"
            )
        except ValueError:
            errors.append((line_number, "invalid date"))
            continue

        try:
            temperature = float(temperature_text)
        except ValueError:
            errors.append((line_number, "invalid temperature"))
            continue

        if not -100.0 <= temperature <= 150.0:
            errors.append((line_number, "temperature out of valid range"))
            continue

        observation_key = (station, date)
        if observation_key in seen_observations:
            errors.append((line_number, "duplicate station/date observation"))
            continue

        seen_observations.add(observation_key)
        observations.setdefault(station, []).append((date, temperature))

for station_observations in observations.values():
    station_observations.sort(key=lambda observation: observation[0])

```

```

    return observations, errors

def station_statistics(observations):
    """Calculate minimum, maximum, and mean temperatures for each station.

    Args:
        observations: A dictionary mapping station names to lists of
            ``(date, temperature)`` tuples.

    Returns:
        A dictionary mapping each station name to a tuple containing its
        minimum temperature, maximum temperature, and mean temperature.
    """
    statistics = {}

    for station, station_observations in observations.items():
        temperatures = [temperature for _, temperature in station_observations]
        minimum = min(temperatures)
        maximum = max(temperatures)
        mean = sum(temperatures) / len(temperatures)
        statistics[station] = (minimum, maximum, mean)

    return statistics

def station_outliers(observations):
    """Return stations whose latest temperature exceeds their mean temperature.

    Args:
        observations: A dictionary mapping station names to date-sorted lists
            of ``(date, temperature)`` tuples.

    Returns:
        A dictionary mapping each outlying station name to a
            ``(date, temperature, mean)`` tuple for its latest observation.
    """
    statistics = station_statistics(observations)

    return {
        station: (station_observations[-1][0], station_observations[-1][1],
                  statistics[station][2])
    }

```

```

        for station, station_observations in observations.items():
            if station_observations[-1][1] > statistics[station][2]
    }

def write_statistics(filename, statistics):
    """Write station temperature statistics to a text file.

    Args:
        filename: Path of the output text file.
        statistics: A dictionary mapping station names to
            ``(minimum, maximum, mean)`` temperature tuples.
    """
    with open(filename, "w", encoding="utf-8") as output_file:
        for station in sorted(statistics):
            minimum, maximum, mean = statistics[station]
            output_file.write(
                f"{station}, {minimum:.1f}, {maximum:.1f}, {mean:.1f}\n"
            )

def main():
    """Read observations, display results, and write station statistics.

    The input filename is read from ``sys.argv[1]`` and the output filename is
    read from ``sys.argv[2]``. The program exits with status code 1 if either
    file cannot be accessed.
    """
    if len(sys.argv) != 3:
        print(f"Usage: {sys.argv[0]} INPUT_FILENAME OUTPUT_FILENAME")
        sys.exit(1)

    input_filename = sys.argv[1]
    output_filename = sys.argv[2]

    try:
        observations, errors = read_observations(input_filename)
    except FileNotFoundError:
        print(f"Error: input file not found: {input_filename}")
        sys.exit(1)
    except (OSError, UnicodeError) as error:
        print(f"Error: could not read input file '{input_filename}': {error}")

```

```

        sys.exit(1)

    for line_number, error_message in errors:
        print(f"Line {line_number}: {error_message}")

    statistics = station_statistics(observations)
    outliers = station_outliers(observations)
    print("Statistics:", statistics)
    print("Outliers:", outliers)

    try:
        write_statistics(output_filename, statistics)
    except FileNotFoundError:
        print(f"Error: output file path not found: {output_filename}")
        sys.exit(1)
    except (OSError, UnicodeError) as error:
        print(f"Error: could not write output file '{output_filename}': {error}")
        sys.exit(1)

if __name__ == "__main__":
    main()

```

Code for test file for unittests:

```

"""Unit tests for the weather station analyzer functions."""

import datetime
import os
import tempfile
import unittest

from p5_Petito_Peter import (
    read_observations,
    station_outliers,
    station_statistics,
    write_statistics,
)

class WeatherStationAnalyzerTests(unittest.TestCase):
    """Test reading, analyzing, and writing weather-station observations."""

```

```

def setUp(self):
    """Create an isolated temporary directory for each test."""
    self.temporary_directory = tempfile.TemporaryDirectory()

def tearDown(self):
    """Remove the temporary directory and its test files."""
    self.temporary_directory.cleanup()

def _write_input_file(self, contents):
    """Create an input file containing the supplied observation text.

    Args:
        contents: Text to write to the temporary input file.

    Returns:
        The path to the created input file.
    """
    path = os.path.join(self.temporary_directory.name, "observations.txt")
    with open(path, "w", encoding="utf-8") as input_file:
        input_file.write(contents)
    return path

def test_read_observations_multiple_stations_and_negative_temperature(self):
    """Read observations for multiple stations, including a negative value."""
    filename = self._write_input_file(
        "North,09:28:09 AM 04/20/2026,-12.5\n"
        "South,09:29:09 AM 04/20/2026,80.0\n"
        "North,09:27:09 AM 04/20/2026,10.0\n"
    )

    observations, errors = read_observations(filename)

    self.assertEqual(errors, [])
    self.assertEqual(set(observations), {"North", "South"})
    self.assertEqual(
        observations["North"],
        [
            (datetime.datetime(2026, 4, 20, 9, 27, 9), 10.0),
            (datetime.datetime(2026, 4, 20, 9, 28, 9), -12.5),
        ],
    )

```

```

def test_read_observations_rejects_duplicate_station_date(self):
    """Reject a second observation with the same station and date."""
    filename = self._write_input_file(
        "North,09:28:09 AM 04/20/2026,70.0\n"
        "North,09:28:09 AM 04/20/2026,71.0\n"
    )

    observations, errors = read_observations(filename)

    self.assertEqual(len(observations["North"]), 1)
    self.assertEqual(errors, [(2, "duplicate station/date observation")])

def test_read_observations_rejects_out_of_range_temperatures(self):
    """Reject temperatures below -100.0 and above 150.0."""
    filename = self._write_input_file(
        "North,09:28:09 AM 04/20/2026,-100.1\n"
        "South,09:28:09 AM 04/20/2026,150.1\n"
        "Valid,09:28:09 AM 04/20/2026,150.0\n"
    )

    observations, errors = read_observations(filename)

    self.assertEqual(observations, {
        "Valid": [(datetime.datetime(2026, 4, 20, 9, 28, 9), 150.0)]
    })
    self.assertEqual(
        errors,
        [
            (1, "temperature out of valid range"),
            (2, "temperature out of valid range"),
        ],
    )

def test_station_statistics_calculates_minimum_maximum_and_mean(self):
    """Calculate accurate minimum, maximum, and mean temperatures."""
    date = datetime.datetime(2026, 4, 20, 9, 28, 9)
    observations = {"North": [(date, -10.0), (date, 10.0), (date, 20.0)]}

    self.assertEqual(station_statistics(observations), {
        "North": (-10.0, 20.0, 20.0 / 3.0)
    })

```

```

def test_station_outliers_returns_latest_temperature_above_mean(self):
    """Return only stations whose latest temperature exceeds their mean."""
    earlier = datetime.datetime(2026, 4, 20, 9, 28, 9)
    latest = datetime.datetime(2026, 4, 21, 9, 28, 9)
    observations = {
        "Outlier": [(earlier, 10.0), (latest, 30.0)],
        "Normal": [(earlier, 30.0), (latest, 10.0)],
    }

    self.assertEqual(
        station_outliers(observations),
        {"Outlier": (latest, 30.0, 20.0)},
    )

def test_write_statistics_orders_stations_and_formats_numbers(self):
    """Write lexicographically sorted stations with one decimal digit."""
    filename = os.path.join(self.temporary_directory.name, "statistics.txt")
    statistics = {
        "Zulu": (-1.0, 2.0, 0.5),
        "Alpha": (1.0, 4.0, 2.5),
    }

    write_statistics(filename, statistics)

    with open(filename, "r", encoding="utf-8") as output_file:
        self.assertEqual(
            output_file.read(),
            "Alpha,1.0,4.0,2.5\nZulu,-1.0,2.0,0.5\n",
        )

def test_read_observations_reports_missing_input_file(self):
    """Raise a clear FileNotFoundError when the input file is absent."""
    filename = os.path.join(self.temporary_directory.name, "missing.txt")

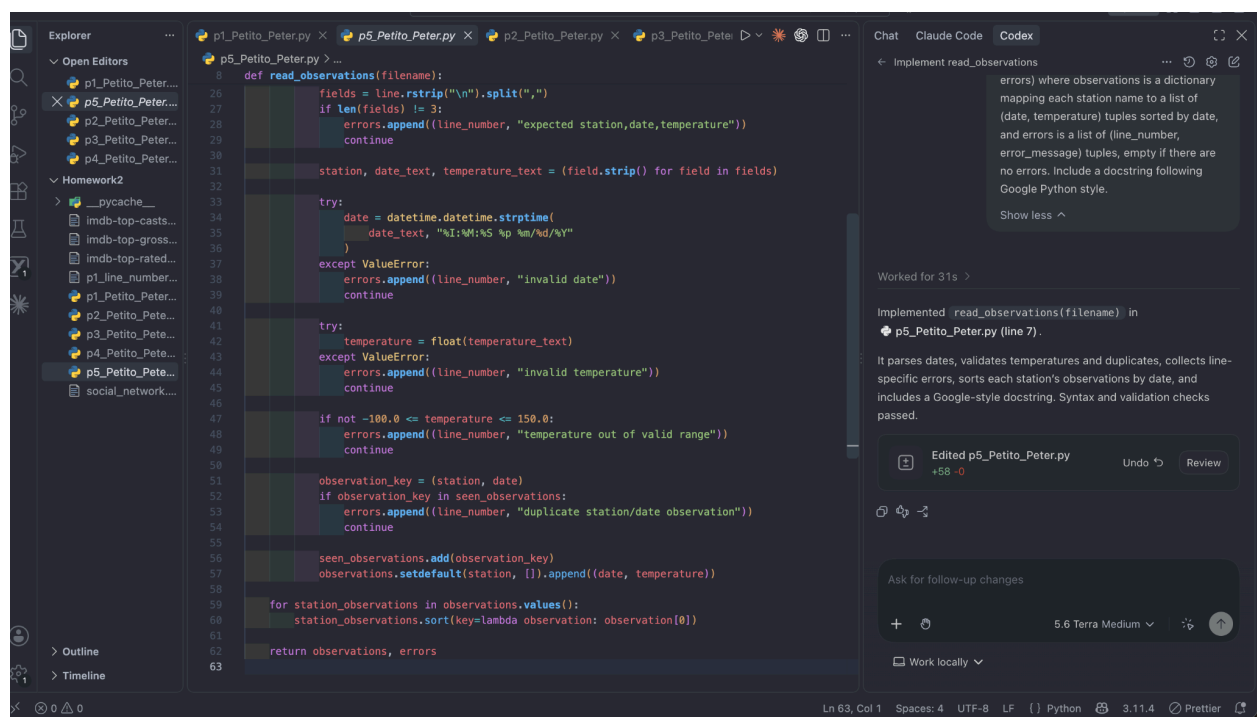
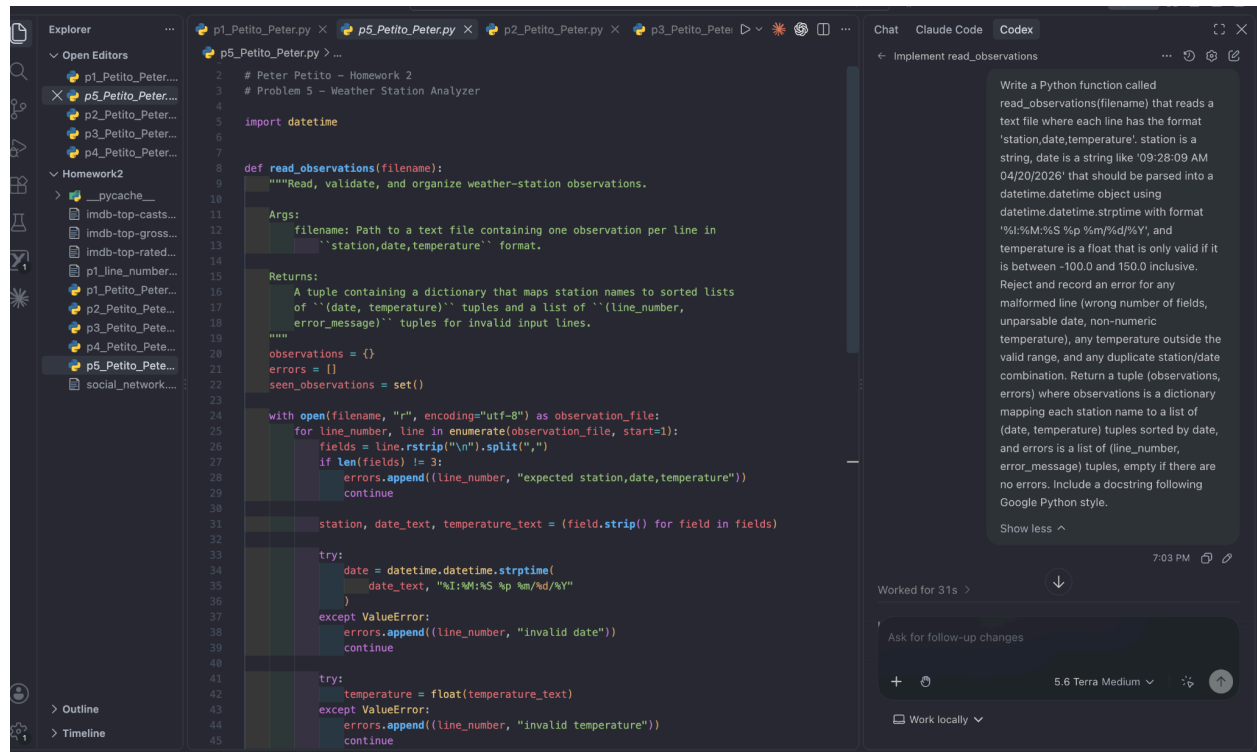
    with self.assertRaises(FileNotFoundError):
        read_observations(filename)

if __name__ == "__main__":
    unittest.main()

```

Prompts:

a)



b)

```
def read_observations(filename):
    """Read observations from a file and return a dictionary of station observations.
    Args:
        filename: The name of the file to read.
    Returns:
        A dictionary mapping station names to lists of (date, temperature) tuples.
    """
    seen_observations = {}
    observations = {}
    errors = []
    with open(filename, 'r') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split(',')
            if len(parts) != 2:
                errors.append((line_number, "duplicate station/date observation"))
                continue
            station, observation = parts
            observation_key = (station, observation)
            if observation_key in seen_observations:
                errors.append((line_number, "duplicate station/date observation"))
                continue
            seen_observations.add(observation_key)
            observations.setdefault(station, []).append((date, temperature))

    for station_observations in observations.values():
        station_observations.sort(key=lambda observation: observation[0])

    return observations, errors

def station_statistics(observations):
    """Calculate minimum, maximum, and mean temperatures for each station.
    Args:
        observations: A dictionary mapping station names to lists of
            `(date, temperature)` tuples.
    Returns:
        A dictionary mapping each station name to a tuple containing its
        minimum temperature, maximum temperature, and mean temperature.
    """
    statistics = {}
    for station, station_observations in observations.items():
        temperatures = [temperature for _, temperature in station_observations]
        minimum = min(temperatures)
        maximum = max(temperatures)
        mean = sum(temperatures) / len(temperatures)
        statistics[station] = (minimum, maximum, mean)

    return statistics
```

Write a Python function called `station_statistics(observations)` that takes the observations dictionary produced by `read_observations` and returns a dictionary mapping each station name to its minimum, maximum, and mean temperature, computed over that station's list of `(date, temperature)` tuples. Include a docstring following Google Python style.

Worked for 18s

Added `station_statistics(observations)` to `p5_Petito_Peter.py` (line 58).

It returns each station's (minimum, maximum, mean) temperature tuple. Syntax and behavior checks passed.

Edited `p5_Petito_Peter.py` +23 -0 Undo Review

7:06 PM

Ask for follow-up changes

5.6 Terra Medium

Work locally

c)

```
def station_statistics(observations):
    """Calculate minimum, maximum, and mean temperatures for each station.
    Args:
        observations: A dictionary mapping station names to lists of
            `(date, temperature)` tuples.
    Returns:
        A dictionary mapping each station name to a tuple containing its
        minimum temperature, maximum temperature, and mean temperature.
    """
    statistics = {}
    for station, station_observations in observations.items():
        temperatures = [temperature for _, temperature in station_observations]
        minimum = min(temperatures)
        maximum = max(temperatures)
        mean = sum(temperatures) / len(temperatures)
        statistics[station] = (minimum, maximum, mean)

    return statistics

def station_outliers(observations):
    """Return stations whose latest temperature exceeds their mean temperature.
    Args:
        observations: A dictionary mapping station names to date-sorted lists
            of `(date, temperature)` tuples.
    Returns:
        A dictionary mapping each outlying station name to a
            `(date, temperature, mean)` tuple for its latest observation.
    """
    statistics = station_statistics(observations)

    return {
        station: (station_observations[-1][0], station_observations[-1][1],
                 statistics[station][2])
        for station, station_observations in observations.items()
        if station_observations[-1][1] > statistics[station][2]
    }
```

Write a Python function called `station_outliers(observations)` that calls `station_statistics(observations)` internally, then uses a dictionary comprehension (not a for-loop) to build and return a dictionary containing only the stations whose latest reported temperature (the temperature from the last date in that station's sorted list) exceeds that station's mean temperature. Map each outlying station name to a tuple `(date, temperature, mean)` representing its latest observation and mean. Include a docstring following Google Python style.

Worked for 14s

Added `station_outliers(observations)` to `p5_Petito_Peter.py` (line 81).

It calls `station_statistics`, uses a dictionary comprehension, and returns only stations whose latest temperature exceeds their mean. Validation passed.

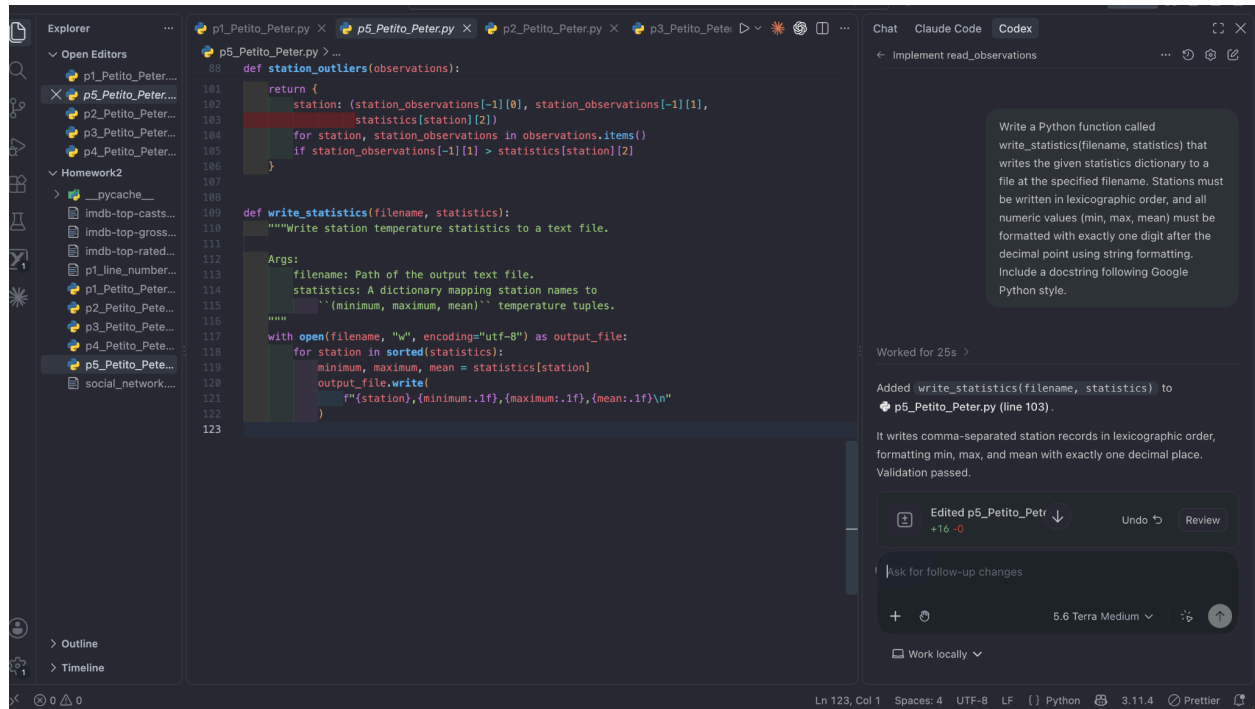
Edited `p5_Petito_Peter.py` Undo Review

Ask for follow-up changes

5.6 Terra Medium

Work locally

d)



```
def station_outliers(observations):
    return {
        station: (station_observations[-1][0], station_observations[-1][1],
                  statistics[station][2])
        for station, station_observations in observations.items()
        if station_observations[-1][1] > statistics[station][2]
    }

def write_statistics(filename, statistics):
    """Write station temperature statistics to a text file.

    Args:
        filename: Path of the output text file.
        statistics: A dictionary mapping station names to
            ``(minimum, maximum, mean)`` temperature tuples.

    """
    with open(filename, "w", encoding="utf-8") as output_file:
        for station in sorted(statistics):
            minimum, maximum, mean = statistics[station]
            output_file.write(
                f"{station},{minimum:.1f},{maximum:.1f},{mean:.1f}\n"
            )
```

Write a Python function called `write_statistics(filename, statistics)` that writes the given statistics dictionary to a file at the specified filename. Stations must be written in lexicographic order, and all numeric values (min, max, mean) must be formatted with exactly one digit after the decimal point using string formatting. Include a docstring following Google Python style.

Worked for 25s >

Added `write_statistics(filename, statistics)` to `p5_Petito_Peter.py` (line 103).

It writes comma-separated station records in lexicographic order, formatting min, max, and mean with exactly one decimal place. Validation passed.

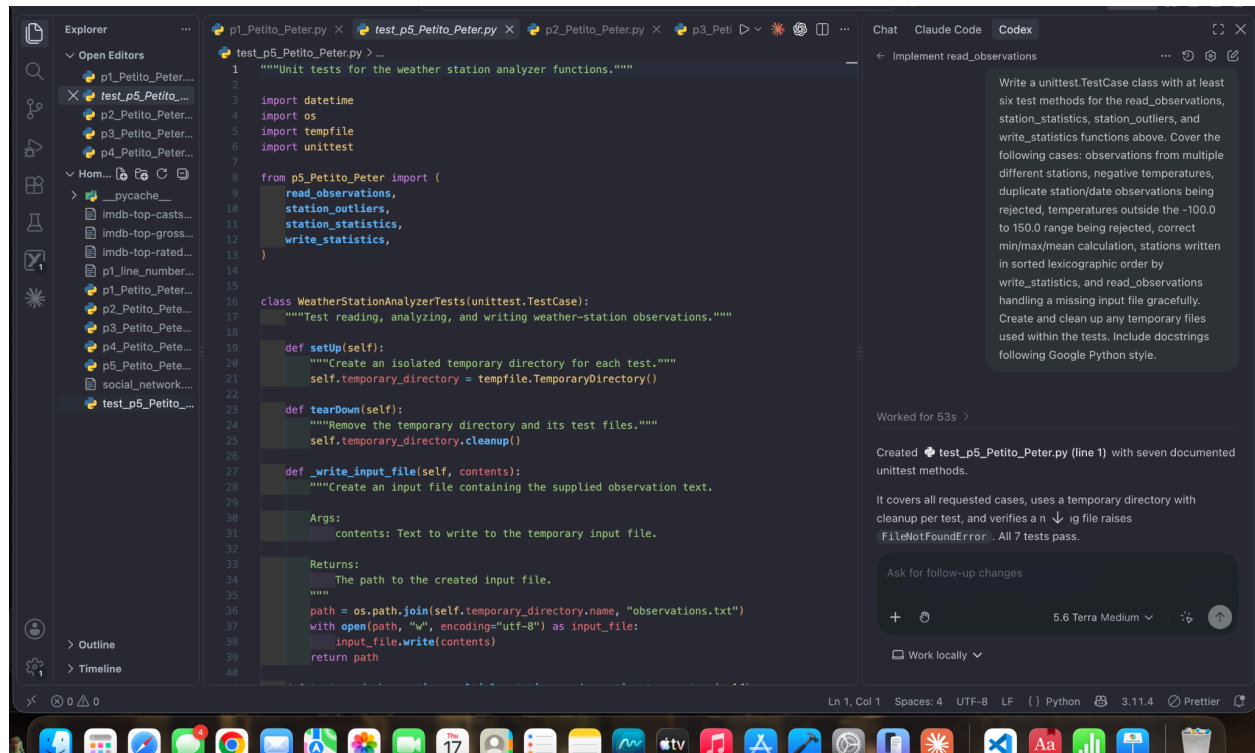
Edited `p5_Petito_Peter.py` +16 -0 Undo Review

Ask for follow-up changes

5.6 Terra Medium

Work locally

e)



```
"""Unit tests for the weather station analyzer functions."""
import datetime
import os
import tempfile
import unittest

from p5_Petito_Peter import (
    read_observations,
    station_outliers,
    station_statistics,
    write_statistics,
)

class WeatherStationAnalyzerTests(unittest.TestCase):
    """Test reading, analyzing, and writing weather-station observations."""

    def setUp(self):
        """Create an isolated temporary directory for each test."""
        self.temp_directory = tempfile.TemporaryDirectory()

    def tearDown(self):
        """Remove the temporary directory and its test files."""
        self.temp_directory.cleanup()

    def _write_input_file(self, contents):
        """Create an input file containing the supplied observation text.

        Args:
            contents: Text to write to the temporary input file.

        Returns:
            The path to the created input file.
        """
        path = os.path.join(self.temp_directory.name, "observations.txt")
        with open(path, "w", encoding="utf-8") as input_file:
            input_file.write(contents)
        return path
```

Write a `unittest.TestCase` class with at least six test methods for the `read_observations`, `station_statistics`, `station_outliers`, and `write_statistics` functions above. Cover the following cases: observations from multiple different stations, negative temperatures, duplicate station/date observations being rejected, temperatures outside the -100.0 to 150.0 range being rejected, correct min/max/mean calculation, stations written in sorted lexicographic order by `write_statistics`, and `read_observations` handling a missing input file gracefully. Create and clean up any temporary files used within the tests. Include docstrings following Google Python style.

Worked for 53s >

Created `test_p5_Petito_Peter.py` (line 1) with seven documented unittest methods.

It covers all requested cases, uses a temporary directory with cleanup per test, and verifies a `FileNotFoundError`. All 7 tests pass.

Ask for follow-up changes

5.6 Terra Medium

Work locally

f)

